Synchronization Tools



Background



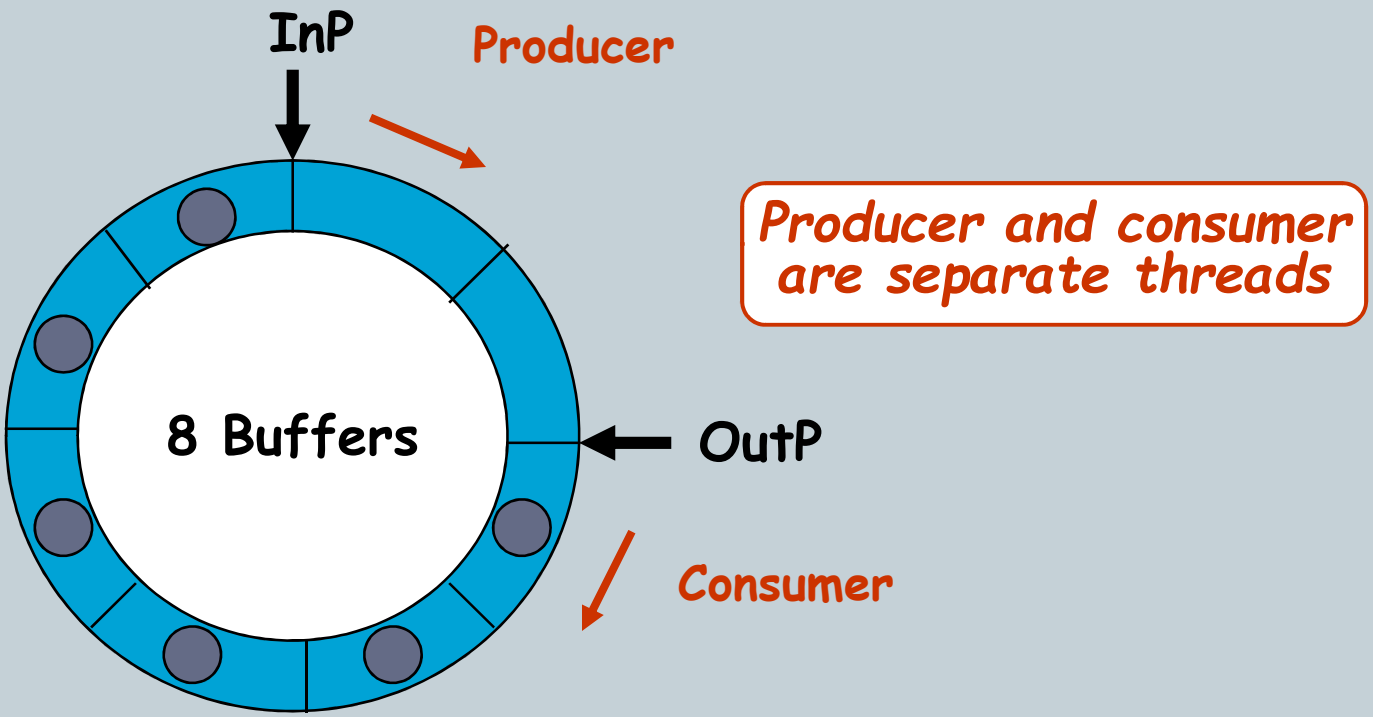
- Processes can execute concurrently
- Concurrent access with shared data
- Need to maintain data consistency

Illustration of the problem

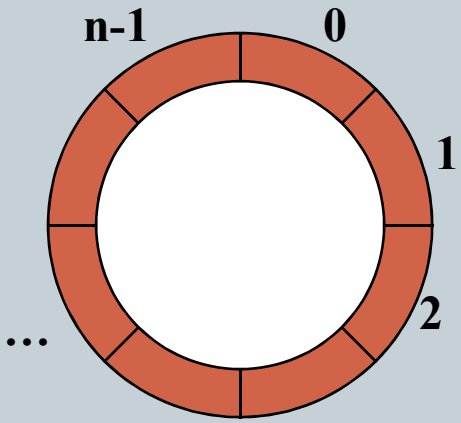


- Shared-memory solution to bounded-buffer problem

Producer consumer problem



Bounded-Buffer




```
#define BUFFER_SIZE = n

typedef struct
{
….
} item ;

item buffer [BUFFER_SIZE] ;
int in = 0; // place to add
int out = 0 ; // place to get
int counter = 0;
```

Producer consumer problem



Producer



```
item next_produced;

while (true)
{
/* produce an item in next produced */

     while (counter == BUFFER_SIZE) ;
     /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Consumer



```
item next_consumed ;

while (true) {
     while (counter == 0) ;
     /* do nothing */
     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;
/* consume the item in next consumed */
}
```

Producer/Consumer with Busy Waiting

Producer Produce

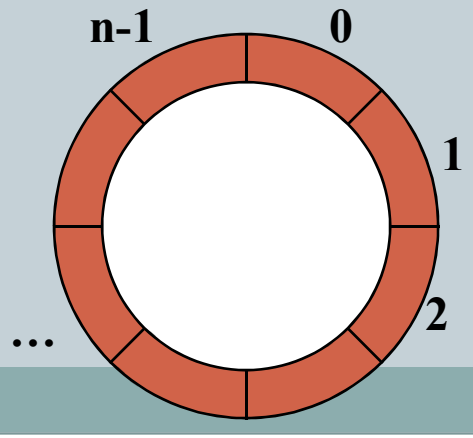
```
while (true)
{
while (counter == BUFFER_SIZE)
        /* do nothing */

buffer[in] = next_produced;
in = (in + 1) % BUFFER_SIZE;
counter++;
}
```

consumer Produce

```
while (true)
{
while (counter == 0) ;
    /* do nothing */

next_consumed = buffer[out];
out = (out + 1) % BUFFER_SIZE
counter--;
}
```




```
Global variables:
    item buffer[ BUFFER_SIZE ]
    int In = 0    // place to add
    int Out = 0   // place to get
    int counter // number of items
```

Bounded Buffer



- The statements

  counter++;
  counter--;

  must be performed *atomically*.

- Atomic operation means an operation that completes in its entirety without interruption.

Bounded Buffer



- The statement "**count++**" may be implemented in machine language as:

  register1 = counter
  register1 = register1 + 1
  counter = register1

- The statement "**count—**" may be implemented as:

  register2 = counter
  register2 = register2 – 1
  counter = register2

Bounded Buffer



- If both the producer and consumer attempt to update the buffer concurrently, the assembly language statements may get interleaved.

- Interleaving depends upon how the producer and consumer processes are scheduled.

Bounded Buffer



- Consider this execution interleaving with "count = 5" initially:

S0: producer execute register1 = counter {register1 = 5}
S1: producer execute register1 = register1 + 1 {register1 = 6}
S2: consumer execute register2 = counter {register2 = 5}
S3: consumer execute register2 = register2 – 1 {register2 = 4}
S4: producer execute counter = register1 {counter = 6 }
S5: consumer execute counter = register2 {counter = 4}

- The value of **count** may be either 4 or 6, where the correct result should be 5.

Race Condition



- **Race condition**: The situation where several processes access – and manipulate shared data concurrently. The final value of the shared data depends upon which process finishes last.

- To prevent race conditions, concurrent processes must be **synchronized**.

Critical Section Problem



- Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section

Critical Section Problem



- **Critical section problem**– ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections

2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely

Solution to Critical-Section Problem



3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

Critical-Section Handling in OS



Two approaches depending on if kernel

- Preemptive or
- Non-preemptive

Initial Attempts to Solve the Problem

- General structure of process P_i

```
do {
    entry section
        critical section
    exit section
        reminder section
} while (1);
```

Algorithm 1

- Only 2 processes, P_0 and P_1

```
do {
    while (turn != i);
        critical section
    turn = j;
        remainder section
  } while (true);
```

- Processes may share some common variables to synchronize their actions.

Algorithm 1



- Shared variables:
  int turn;
  initially **turn = 0**
- Process P_i

```
do {
    while (turn != i) ;
        critical section
    turn = j;
        reminder section
} while (1);
```

- Satisfies mutual exclusion, but not progress
- if *turn* = *0* and P1 is ready to enter its critical section, P1 cannot do so, even though P0 may be in its remainder section.

Algorithm 2



- replace the variable *turn* with the boolean array *flag[2]*.

Algorithm 2



- Shared variables
  boolean flag[2];
  initially **flag [0] = flag [1] = false**
- **flag [i] = true** $\Rightarrow P_i$ ready to enter its critical section

Process P_i

```
do {
        flag[i] = true;
        while (flag[j]);
                critical section
                flag[i] = false;
                remainder section
} while (1);
```

Algorithm 2

- Satisfies mutual exclusion, but not progress requirement.
- To illustrate this problem, we consider the following execution sequence:
  - T0: P0 sets flag[0] = true
  - T1: P1 sets flag[1] = true

  Now *P0* and *P1* are looping forever in their respective *while* statements.

Peterson's Solution

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j);
        critical section
    flag[i] = false;
        remainder section
} while (true);
```

- Combined shared variables of algorithms 1 and 2

Peterson's Solution

- Shared variables:
  int turn;
  initially **turn = 0** or **1**
  initially **flag [0] = flag [1] = false**

Process P_i

```
do {
        flag [i]:= true;
        turn = j;
        while (flag [j] and turn == j);
                critical section
                flag [i] = false;
                remainder section
} while (1);
```

- j =1 – i.

Peterson's Solution

- Provable that the three CS requirement are met:
  1. Mutual exclusion is preserved

     P_i enters CS only if:

     either `flag[j] = false` or `turn = i`
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met

Synchronization Hardware

- Peterson's are not guaranteed to work on modern computer architectures
- Solutions based on **locking** mechanism
- Modern machines provide special atomic hardware instructions (**Atomic** = non-interruptible)

```
do {
        acquire lock

          critical section

        release lock

          remainder section

  } while (TRUE);
```

Synchronization Hardware

- simple hardware instructions can be used effectively in solving the critical-section problem, in a **single-processor environment.**
- Current sequence of instructions would be allowed to execute in order without preemption.

Synchronization Hardware



```
boolean test_and_set (boolean *target)
   {
     boolean rv = *target;
     *target = TRUE;
     return rv:
    }
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

Solution using test_and_set()

Shared Boolean variable "**lock**", initialized to **FALSE**

```
do {
    while (test_and_set(&lock)) ;
     /* critical section */
     lock = false;
     /* remainder section */
} while (true);
```

compare_and_swap Instruction

```
int compare _and_swap
    (int *value, int expected, int new_value)
 {
        int temp = *value;

        if (*value == expected)
           *value = new_value;
     return temp;
 }
```

1. Executed atomically
2. Returns the original value of passed parameter “value”
3. Set the variable “value” to “new_value”
   only if “value” ==“expected (under condition).

Solution using compare_and_swap

- Shared integer "**lock**" initialized to **0**;

```
do
{
 while(compare_and_swap(&lock,0,1)!=0);
    /* critical section */
 lock = 0;
     /* remainder section */
} while (true);
```

Bounded-waiting Mutual Exclusion with test_and_set

```
do {
   waiting[i] = true;
   key = true;
   while (waiting[i] && key)
      key = test_and_set(&lock);

   waiting[i] = false;
   /* critical section */

   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;
   if (j == i)
      lock = false;
   else
      waiting[j] = false;
   /* remainder section */
} while (true);
```

Mutex Locks



- Previous solutions are complicated and generally inaccessible to application programmers
- Simplest is **mutex** lock
- Protect a critical section by first **`acquire()`** a lock then **`release()`** the lock
- Usually implemented via hardware atomic instructions
- Solution requires busy waiting (call "spinlock")

acquire() and release()



```
acquire() {
    while (!available) ; /* busy wait */
    available = false;;
}

release() {
    available = true;
}

do {
acquire lock
    critical section
release lock
  remainder section
} while (true);
```

Semaphore



- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - **`wait()`** and **`signal()`**

Semaphore



```
wait(S) {
     while (S <= 0); // busy wait
     S--;
}

signal(S) {
     S++;
}
```

Semaphore Usage



- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can solve various synchronization problems

Semaphore Usage

- Consider P_1 and P_2 that require S_1 to happen before S_2
  Create a semaphore "**synch**" initialized to 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

- Can implement a counting semaphore S as a binary semaphore

Semaphore Implementation



- no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time
- critical section problem , **`wait`** and **`signal`** code are placed in the critical section
  - **busy waiting** in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- Applications may spend lots of time in critical sections
- Therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- Using waiting queue and it has
  - value (of type integer)
  - pointer to next record in the list

- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

Semaphore Implementation with no Busy waiting

```
typedef struct{
  int value;
  struct process *list;
  } semaphore;
```

Semaphore Implementation with no Busy waiting

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Deadlock and Starvation



Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

P_0

```
wait(S);
wait(Q);
   ...
signal(S);
signal(Q);
```

P_1

```
wait(Q);
wait(S);
...
signal(Q);
signal(S);
```

Deadlock and Starvation



- **Starvation – indefinite blocking**
  - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**